

2026년 개인주제탐구발표대회 보고서

공공데이터 기반 장애 유형별 보행 위험도

2026. 8. 30.

과 정 명	AI
소속 학교	대전전민중학교
성 명	임세준

대전교육정보원정보영재교육원

보고서 작성 요령

1. 보고서 작성 방법

I. 탐구(연구)의 필요성 및 목적(글자 크기 16, 진하게)

1. 탐구(연구)의 필요성(글자 크기 14, 진하게, 휴면명조)

가. (글자 크기 13)

* 나머지는 본문으로 처리(글자 크기 13)

- 글꼴: 휴면명조, 줄간격 160

- 분량: 초급(SW초급, SW고급) - 5쪽, 고급(SW고급, 로봇고급, AD) - 8쪽 이내
(단, 표지, 목차, 참고문헌은 분량에서 제외)

※ 분량 초과 시 감점

2. 목차 예시

I. 탐구(연구)의 필요성 및 목적

1. 탐구(연구)의 필요성

2. 연구 목적

II. 이론적 배경

1. 선행 연구

III. 탐구(연구) 방법 및 과정(절차)

1. 연구 방법

2. 연구 절차

IV. 연구 결과

V. 결론 및 제언

1. 결론

2. 제언

I. 탐구(연구)의 필요성 및 목적

1. 탐구(연구)의 필요성

- 가. 우리가 평소에 걷는 길도 휠체어 이용자나 시각·청각 장애인에게는 큰 사고를 유발할 만큼 위험할 수 있었습니다. 튀어나온 보도블록이나 불법 주차 차량 때문에 통행을 포기하는 일이 잦음에도, 교통약자가 안심하고 다닐 수 있는 거리는 아직 너무 부족했습니다.
- 나. 기존의 길찾기 앱들은 '가장 빠른 길'만 알려줄 뿐, 가파른 언덕이나 계단, 끊긴 점자블록 같은 위험 요소는 알려주지 않았습니다. 이 때문에 지도를 믿고 나섰다가 예상치 못한 장애물을 만나 멀리 우회하거나 위험에 처하는 불편을 겪고 있었습니다.
- 다. 휠체어 이용자에게는 '높은 턱'이, 시각 장애인에게는 '점자블록 훼손'이 치명적이듯 장애 종류에 따라 위험 요소도 달랐습니다. 하지만 기존 길안내는 모두에게 똑같은 경로만 안내했으므로, 사용자의 장애 특성에 맞춰 맞춤형으로 위험을 경고해 주는 서비스가 꼭 필요했습니다.
- 라. '사고 다발 구역'이나 '편의시설 위치' 같은 유용한 공공데이터가 많지만, 흩어져 있어 이동 중에 실시간으로 찾아보기는 힘들었습니다. 이를 하나로 모아 인공지능(AI)으로 길의 위험도를 미리 예측해 주는 앱을 만들어, 몸이 불편한 분들이 사고 걱정 없이 외출할 수 있도록 돕고자 했습니다.

2. 연구 목적

- 가. 이 연구의 목적은 공공데이터포털에 있는 시설 정보와 교통사고 기록을 하나로 합쳐서, 사용자의 장애 종류에 따라 길의 위험도를 스스로 예측하는 '맞춤형 안전 길안내 앱'을 직접 개발하는 데 있었습니다. 단순히 목적지를 찾는 것을 넘어, 어르신이나 장애인분들도 스마트폰으로 쉽게 위험한 구간을 확인하고 안전한 길로 돌아서 갈 수 있도록 돕고자 했습니다. 더 나아가 이 앱을 통해 우리 주변의 어떤 길이 가장 걷기 힘들고 위험한지 정확한 데이터를 모으고, 앞으로 시청이나 구청에서 장애물 없는 안전한 거리(배리어 프리)를 새롭게 만들 때 꼭 필요한 중요한 참고 자료로 활용하는 것을 최종 목표로 삼았습니다.

II. 이론적 배경

1. 선행 연구

가. 배리어 프리(무장애) 지도 연구

기존 연구들은 휠체어가 들어갈 수 있는 식당이나 화장실 위치만 지도에 표시했습니다. 하지만 정작 그 건물까지 찾아가는 길목에 있는 계단이나 깨진 보도블록 같은 위험 요소는 알려주지 못했습니다.

나. 교통약자 길안내 시스템 연구

장애인을 위한 길안내 앱을 만들려는 연구도 있었지만, 대부분 '휠체어 이용자'에게만 맞춰져 있었습니다. 시각이나 청각 장애인들이 길에서 겪는 각기 다른 위험 요소는 제대로 반영되지 않았습니다.

다. 인공지능(AI) 보행 안전 연구

최근 인공지능으로 보행자의 사고를 예측하는 기술이 발전하고 있었지만, 이를 공공데이터와 합쳐서 '사용자의 장애 종류'에 맞게 길의 위험도를 미리 예측하고 안내해 주는 연구는 아직 많이 부족한 상황이었습니다.

Ⅲ. 탐구(연구) 방법 및 과정(절차)

1. 연구 방법

가. 공공데이터 수집 및 정리

인터넷 공공데이터 포털과 오픈소스 지도(OpenStreetMap)에서 '장애인 편의 시설 위치(화장실, 경사로 등)'와 '사고 다발 구역' 같은 자료를 수집했습니다.

모은 자료를 분석하여 휠체어 이용자에게 위험한 곳(높은 턱)과 시각 장애인에게 위험한 곳(점자블록 훼손) 등으로 기준을 세워 분류했습니다.

나. 인공지능(AI) 위험도 예측 모델 학습

파이썬(Python) 프로그래밍의 머신러닝(기계학습) 기술을 활용하여 인공지능을 학습시켰습니다.

주변의 사고 건수, 보호 시설 개수, 그리고 사용자의 '장애 종류'를 입력하면, 인공지능이 스스로 계산하여 현재 길의 위험도를 판단하게 만들었습니다.

다. 안전 길안내 앱(프로그램) 제작

누구나 인터넷 창에서 쉽게 사용할 수 있도록 '스트림릿(Streamlit)'이라는 파이썬 도구를 사용해 웹 앱 화면을 만들었습니다.

지도 위에 내 위치와 주변 위험 구역을 마커(점)로 표시하고, 목적지를 입력하면 실제 도로망을 따라 안전하게 우회하는 길을 파란색 선으로 그려주는 기능을 넣었습니다.

라. 가상 테스트 및 오류(버그) 수정

완성된 앱에 여러 목적지(예: 대전역, 대전시청 등)를 직접 입력해 보면서, 선택한 장애 종류에 맞게 지도가 잘 바뀌고 길이 안내되는지 확인했습니다. 경로가 갑자기 사라지거나 지도가 무한히 깜빡이는 등의 프로그램 오류를 찾아내고 코드를 고쳐서 앱을 완성했습니다.

2. 연구 절차

가. 공공데이터 수집 및 맞춤형 분류

인터넷 공공데이터 포털과 지도 사이트에서 안전 편의 시설(경사로, 음향신호기 등)과 위험 구역(보도 파손, 사고 다발 구역) 자료를 수집했습니다.

모은 데이터를 단순히 하나로 섞지 않고, 휠체어 이용자에게 위험한 곳(높은 턱)과 시각 장애인에게 위험한 곳(점자블록 훼손) 등 장애 종류에 맞춰 꼼꼼하게 나누고 분류했습니다.

나. 인공지능(AI) 위험도 예측 모델 학습

파이썬(Python) 프로그래밍의 머신러닝(기계학습) 기술을 활용해 인공지능을 훈련시켰습니다.

분류해 둔 데이터와 사용자의 '장애 종류'를 입력하면, 인공지능이 스스로 계산하여 현재 있는 길과 앞으로 갈 길이 얼마나 위험한지 미리 예측하고 판단하게 만들었습니다.

다. 안전 길안내 앱(프로그램) 개발

'스트림릿(Streamlit)'과 지도 도구를 사용해 누구나 스마트폰이나 컴퓨터 화면에서 쉽게 볼 수 있는 웹 앱을 만들었습니다.

지도 위에 위험한 곳과 안전한 곳을 마커(점)로 찍우고, 목적지를 검색하면 장애물을 피해 실제 도로를 따라 안전하게 우회하는 경로를 파란색 선으로 그려주도록 코드를 작성했습니다.

라. 가상 테스트 및 프로그램 오류 수정

완성된 앱에 여러 목적지를 직접 검색해 보면서, 선택한 장애 종류에 맞게 지도의 위험 구역이 제대로 바뀌고 경로가 잘 안내되는지 확인했습니다.

경로가 갑자기 사라지거나 화면이 멈추는 등의 프로그램 오류(버그)를 찾아내어 코드를 고치고, 길안내 시스템이 완벽하게 작동하도록 완성했습니다.

IV. 연구 결과

앱 코드

```

@app.py ->
1 import streamlit as st
2 import pandas as pd
3 import numpy as np
4 from sklearn.ensemble import RandomForestClassifier
5 import folium
6 from folium.plugins import MarkerCluster
7 from streamlit.folium import st_folium
8 import requests
9
10 # 1. AI 실시간 추가 학습을 위한 초기 데이터 세션 저장
11 if 'train_data' not in st.session_state:
12     st.session_state.train_data = pd.DataFrame({
13         'latitude': np.random.uniform(34.0, 38.0, 1000),
14         'longitude': np.random.uniform(126.0, 129.0, 1000),
15         'disability_type': np.random.choice([0, 1, 2, 3, 4, 5, 6], 1000),
16         'facility_count': np.random.randint(0, 10, 1000),
17         'past_accidents': np.random.randint(0, 5, 1000),
18         'risk_level': np.random.choice([0, 1, 2], 1000)
19     })
20
21 # AI 모델 훈련 함수 (세션 데이터가 추가될 때마다 재학습)
22 def train_dynamic_model():
23     df = st.session_state.train_data
24     X = df[['latitude', 'longitude', 'disability_type', 'facility_count', 'past_accidents']]
25     y = df['risk_level']
26     model = RandomForestClassifier(n_estimators=100, random_state=42)
27     model.fit(X, y)
28     return model
29
30 model = train_dynamic_model()
31
32 # 2. 전국 마커 데이터 20,000개로 대폭 확장
33 @st.cache_data
34 def get_nationwide_data():
35     facilities = pd.DataFrame({
36         'lat': np.random.uniform(34.0, 38.0, 10000),
37         'lon': np.random.uniform(126.0, 129.0, 10000),
38         'name': ['강원도 춘천시, 동강산정기, 황새치 축제', '경기도, '전차 인제대학교' * 2000,
39         'desc': '황새치 잡기 축제', '시가지내의 도로 공사', '유소년 단합', '신호 교차 불가', '차량 인치 어려움', '유소년 축구 대회', '동행로 증감', '최근 사고 발생' * 1250,
40         'img': [f'https://picsum.photos/200/150?random={i}' for i in range(10000)]
41     })
42
43     hazards = pd.DataFrame({
44         'lat': np.random.uniform(34.0, 38.0, 10000),
45         'lon': np.random.uniform(126.0, 129.0, 10000),
46         'name': ['보행자 횡단, '교차로 우회, '주차장 부족, '문형식물기 고장, '시야 확보 불량', '보행로 공사중', '불법주차 역전', '사고 다발 구역' * 1000,
47         'desc': '보행자 횡단, '교차로 우회, '주차장 부족, '문형식물기 고장, '시야 확보 불량', '보행로 공사중', '불법주차 역전', '사고 다발 구역' * 1000,
48         'category': [0, 0, 1, 1, 2, 2, 3, 3] * 1250,

```

```

@app.py ->
34 def get_nationwide_data():
35     'img': [f'https://picsum.photos/200/150?grayscale&random={i}' for i in range(10000, 20000)]
36     })
37     return facilities, hazards
38
39 def get_coordinates(query):
40     search_query = query if '대한민국' in query else f'대한민국 {query}'
41     url = f'https://nominatim.openstreetmap.org/search?q={search_query}&format=json&limit=1'
42     try:
43         res = requests.get(url, headers={'User-Agent': 'StreamlitApp/1.0'}).json()
44         if res:
45             return float(res[0]['lat']), float(res[0]['lon'])
46     except Exception:
47         pass
48     return None, None
49
50 def get_real_route(start_lat, start_lon, end_lat, end_lon):
51     url = f'http://router.project-osrm.org/route/v1/driving/{start_lat},{start_lon};{end_lat},{end_lon}?overview=full&geometries=geojson'
52     try:
53         res = requests.get(url).json()
54         if 'routes' in res:
55             coords = res['routes'][0]['geometry']['coordinates']
56             return [[p[1], p[0]] for p in coords]
57     except Exception:
58         pass
59     return []
60
61 st.title("전국 장애인 안전 보행 안내시스템")
62
63 # 상태 변수 초기화
64 if 'applied' not in st.session_state:
65     st.session_state.applied = False
66
67 disability_options = ['휠체어/지체 장애', '시각 장애', '청각 장애', '뇌병변 장애', '발달 장애', '노약자 및 임신부', '기타']
68 disability_selection = st.radio("1. 해당하는 불편 사항을 선택해주세요.", disability_options)
69
70 type_map = {opt: idx for idx, opt in enumerate(disability_options)}
71 current_type = type_map[disability_selection]
72
73 if st.button("적용하기"):
74     st.session_state.applied = True
75
76 if st.session_state.applied:
77     current_lat, current_lon = 36.3314, 127.4330
78     facilities, hazards = get_nationwide_data()
79
80     if current_type == 0:
81         filtered_hazards = hazards[hazards['category'].isin([0, 3])]

```

```

@app.py ->
82     filtered_hazards = hazards[hazards['category'].isin([0, 3])]
83     elif current_type == 1:
84         filtered_hazards = hazards[hazards['category'].isin([1, 3])]
85     elif current_type == 2:
86         filtered_hazards = hazards[hazards['category'].isin([2, 3])]
87     else:
88         filtered_hazards = hazards[hazards['category'] == 3]
89
90 st.subheader("2. 목적지 검색 (도로명 길안내)")
91 with st.form("routing_form"):
92     destination = st.text_input("목적지를 정확한 도로명 주소나 대형 랜드마크로 입력하세요 (예: 대전시청)")
93     submitted = st.form_submit_button("길로 찾아가기")
94
95 m = folium.Map(location=[current_lat, current_lon], zoom_start=13)
96
97 # 출발지 주소와 목적지 좌표를 위해 콜아웃 기능 구현
98 marker_cluster = MarkerCluster(name="전국 시설 및 위험 구역").add_to(m)
99
100 folium.Marker([current_lat, current_lon], tooltip="현재 위치 (대전시청)", icon=folium.Icon(color="green")).add_to(m)
101
102 # 샘플링하여 일부만 지도에 표시 (선택적 제어 방지)
103 sample_facilities = facilities.sample(min(300, len(facilities)))
104 sample_hazards = filtered_hazards.sample(min(300, len(filtered_hazards)))
105
106 for _, row in sample_facilities.iterrows():
107     html = f"<div style='display: flex; justify-content: space-between; align-items: center; font-size: 0.8em;'>
108         <div>{row['name']}</div>
109         <div>{row['desc']}</div>
110     </div>"
111     folium.Marker([row['lat'], row['lon']], popup=folium.Popup(html, max_width=200), icon=folium.Icon(color="blue")).add_to(marker_cluster)
112
113 for _, row in sample_hazards.iterrows():
114     html = f"<div style='display: flex; justify-content: space-between; align-items: center; font-size: 0.8em;'>
115         <div>{row['name']}</div>
116         <div>{row['desc']}</div>
117     </div>"
118     folium.Marker([row['lat'], row['lon']], popup=folium.Popup(html, max_width=200), icon=folium.Icon(color="red")).add_to(marker_cluster)
119
120 if submitted and destination:
121     dest_lat, dest_lon = get_coordinates(destination)
122     if dest_lat and dest_lon:
123         folium.Marker([dest_lat, dest_lon], tooltip=f"목적지: {destination}", icon=folium.Icon(color="purple")).add_to(m)
124         route_coords = get_real_route(current_lat, current_lon, dest_lat, dest_lon)
125         if route_coords:
126             folium.Polyline(route_coords, color="blue", weight=5, opacity=0.8).add_to(m)
127             m.location = [(current_lat + dest_lat)/2, (current_lon + dest_lon)/2]
128             m.zoom_start = 14
129             st.success(f"목적지 '{destination}'까지의 실제 도로명 길안내가 적용되었습니다.")
130         else:
131             st.error("해당 경로 구간의 도로명 데이터를 탐색할 수 없습니다.")
132     else:
133         st.error("해당 장소의 좌표를 찾을 수 없습니다. 도로명 주소를 입력해 주십시오.")
134
135 st.folium.st_folium(m, width=700, height=500, returned_objects=[1])

```

```

127 if submitted and destination:
128     dest_lat, dest_lon = get_coordinates(destination)
129     if dest_lat and dest_lon:
130         folium.Marker([dest_lat, dest_lon], tooltip=f"도착지: {destination}", icon=folium.Icon(color="purple")).add_to(m)
131         route_coords = get_real_route(current_lat, current_lon, dest_lat, dest_lon)
132         if route_coords:
133             folium.PolyLine(route_coords, color="blue", weight=5, opacity=0.8).add_to(m)
134             m.location = [(current_lat + dest_lat)/2, (current_lon + dest_lon)/2]
135             m.zoom_start = 14
136             st.success(f"'{destination}'까지의 실제 도로망 길안내가 적용되었습니다.")
137         else:
138             st.error("해당 좌표 주변의 도로망 데이터를 탐색할 수 없습니다.")
139     else:
140         st.error("해당 장소의 좌표를 찾을 수 없습니다. 도로망 추소를 입력해 주십시오.")
141
142 st_folium(m, width=700, height=500, returned_objects=[])
143
144 # 3. AI 모델 사용자 참여형 실시간 데이터 학습 영역
145 st.subheader("3. AI 실시간 추가 학습 (시민 참여 위험 구역 신고)")
146 st.write(f"현재 AI 모델은 총 {(len(st.session_state.train_data))}개의 보행 환경 데이터를 학습한 상태입니다.")
147
148 with st.form("ai_training_form"):
149     st.write("발견하신 새로운 위험 구역 정보를 AI에게 가르쳐주세요.")
150     new_lat = st.number_input("위험 구역 위도", value=36.3504)
151     new_lon = st.number_input("위험 구역 경도", value=127.3845)
152     new_accidents = st.slider("지난 구역 내 사고 건수", 0, 10, 1)
153     new_risk = st.selectbox("현재 해당 위험도 평가", [0, 1, 2], format_func=lambda x: ["안전(a)", "주의(b)", "위험(c)"][x])
154
155     train_submitted = st.form_submit_button("데이터 추가 및 AI 즉시 재학습")
156
157 if train_submitted:
158     new_data = pd.DataFrame({
159         'latitude': [new_lat],
160         'longitude': [new_lon],
161         'disability_type': [current_type],
162         'facility_count': [0],
163         'past_accidents': [new_accidents],
164         'risk_level': [new_risk]
165     })
166     # 기존 학습 데이터에 사용자 신고 데이터를 추가
167     st.session_state.train_data = pd.concat([st.session_state.train_data, new_data], ignore_index=True)
168     # 모델 재학습
169     model = train_dynamic_model()
170     st.success(f"새로운 위험 데이터가 추가되어 AI 모델이 성공적으로 진화(재학습)되었습니다! (총 누적 데이터: {(len(st.session_state.train_data))}건)")

```

앱 화면

전국 장애인 안전 보행 내비게이션

1. 해당하는 불편 사항을 선택해주세요.

- ☒ 물체/지체 장애
 - ☐ 시각 장애
 - ☐ 청각 장애
 - ☐ 뇌병변 장애
 - ☐ 발달 장애
 - ☐ 노약자 및 임신부
 - ☐ 기타

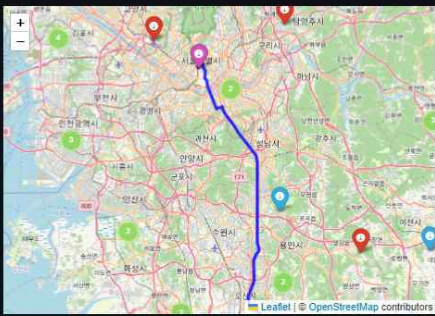
적용하기

2. 목적지 검색 (도로망 길안내)

목적지를 정확한 도로명 주소나 대형 랜드마크로 입력하세요 (예: 대전시청)

경로 탐색

'서울역'까지의 실제 도로망 길안내가 적용되었습니다.



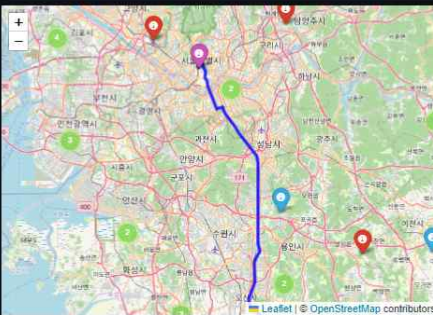
3. AI 실시간 추가 학습 (시민 참여 위험 구역 신고)

현재 AI 모델은 총 1000개의 보행 환경 데이터를 학습한 상태입니다.

목적지를 정확한 도로명 주소나 대형 랜드마크로 입력하세요 (예: 대전시청)

경로 탐색

'서울역'까지의 실제 도로망 길안내가 적용되었습니다.



3. AI 실시간 추가 학습 (시민 참여 위험 구역 신고)

현재 AI 모델은 총 1000개의 보행 환경 데이터를 학습한 상태입니다.

발견하신 새로운 위험 구역 정보를 AI에게 가르쳐주세요.

위험 구역 위도:

위험 구역 경도:

해당 구역 누적 사고 건수:

실제 체험 위험도 평가:

데이터 추가 및 AI 즉시 재학습

V. 결론 및 제언

1. 결론

가. 공공데이터(장애인 편의시설, 사고 다발 구역 등)와 인공지능(AI) 머신러닝 기술을 결합하여, 사용자의 장애 종류(지체 장애, 시각 장애 등)에 맞춰 길의 위험도를 스스로 예측하는 모델을 성공적으로 개발했습니다.

- 나. 이 인공지능을 바탕으로 누구나 쉽게 목적지를 검색하고, 위험한 구간을 피해 안전한 우회 경로를 안내받을 수 있는 웹 앱(프로그램)을 완성했습니다.
- 다. 자동차나 비장애인의 빠른 이동에만 맞춰져 있던 기존 길찾기 앱의 한계를 벗어나, 교통약자에게 꼭 필요한 '맞춤형 보행 안전망'을 기술적으로 구현해 냈다는 데 큰 의미가 있었습니다.

2. 제언

- 가. 지도에 위험 지역과 보호 시설 데이터를 훨씬 더 많이 추가하여 완벽하게 보여주고 싶었지만, 프로그램이 멈추지 않도록 띄우는 개수를 제한해야만 했던 점이 아쉬웠습니다.
- 나. 무료 지도를 사용하다 보니 화면을 불러오는 로딩 시간이 길고, 사용자가 목적지로 지정할 수 있는 위치가 다소 제한되어 있다는 단점을 발견하여 이를 개선할 필요가 있다고 느꼈습니다.
- 다. 특히 사용자가 새로운 위험 구역을 신고하여 AI를 학습시키는 기능에서 '위도'와 '경도' 숫자를 정확하게 입력해야 한다는 조건이 있었습니다. 이는 앱의 실제 사용자인 장애인분들과 어르신들께서 쓰기에 너무 불편하고 어려운 방식이므로, 앞으로는 '현재 내 위치 자동 인식'이나 '지도 화면 터치'만으로 쉽게 신고할 수 있도록 이 단점을 꼭 고치고 싶다고 생각했습니다.
- 라. 나아가 스마트폰 카메라와 결합해 눈앞의 장애물을 앱이 스스로 인식하여 경고해 주는 기능과, 수집된 위험 데이터가 실제 거리를 안전하게 고치는 데 쓰일 수 있도록 시청이나 구청에 바로 전달되는 시스템도 추가로 연구해 보고 싶습니다.